Lumen

Understand Lumen and answer any question

This guide is for you, the person who has to **explain** the project. It is written in plain words. Each question has a **short answer you can say out loud**, and a **"if they ask deeper"** part for follow-ups.

Repository: github.com/salman-kv/lumen
Version of this document: 10 November 2026

Contents

Part 1. The project in plain words . . . 3

The 30-second version . . . 3

The 2-minute version . . . 3

What problem does it solve? . . . 3

Part 2. Mental pictures (use these to explain) . . . 3

Part 3. How it works from start to finish . . . 4

Step 1: an organization signs up . . . 4

Step 2: they upload a PDF . . . 4

Step 3: a visitor asks a question . . . 4

Part 4. Questions and answers . . . 5

A. The project . . . 5

B. Architecture . . . 5

C. Login, security and users . . . 6

D. Multi-tenancy and data . . . 7

E. The AI . . . 7

F. Ollama, Groq and the language model . . . 8

G. Backend implementation . . . 9

H. Frontend . . . 9

I. Hosting and deployment . . . 10

J. Testing and quality . . . 10

Part 5. Hard questions and honest answers . . . 11

Part 6. Things to say when you do not know something . . . 11

Part 7. Files to open before the interview (read in this order, 10 minutes each) . . . 11

Part 8. Cheat sheet (numbers and names) . . . 12

How to use it

1. Read Part 1 to 3 once, slowly. They give you the whole picture.
2. Read Part 4 (the questions) a few times. Say the answers out loud in your own words. Do not memorise them.
3. Read Part 5 (hard questions) last. Honest answers about limits impress interviewers more than perfect-sounding ones.
4. Open the code files listed in Part 7 and look at them once. Then you can say "let me show you".

The most important rule: never bluff. If you do not know something, say "I did not build that part, but here is how I would do it". The guide tells you where the real limits are.

Part 1. The project in plain words

The 30-second version

"Lumen lets an organization, like a university or a company, upload its policy PDFs and get a chat assistant for its students or staff. People ask questions and get answers that show the exact document and page. When a question needs a decision, like 'am I eligible for this scholarship?', the system breaks it into small checks, decides each one, combines the results, and then explains the answer. Every organization's data is completely separate."

The 2-minute version

"I built a multi-tenant AI assistant. A tenant is one organization. The organization signs up, uploads up to ten PDFs, and gets a public link and an embeddable chat bubble.

The key idea is that I do not let the language model decide everything, because language models can make things up. So there are three layers. The **language model** only understands wording and writes the final sentence. **Small specialised models** do the real decisions: search, question type, and checking each rule. **Normal application code** routes the message, combines the decisions, and refuses to answer when it has no evidence.

For example, 'am I eligible for the Merit Scholarship?' becomes several yes/no checks: year of study, GPA, and no other scholarship. Each check is evaluated separately and in parallel. The code combines them: all must pass. Then the language model only explains the finished result. It cannot change the verdict.

The backend is Python with FastAPI, the data is in SQLite, the frontend is React, and the model is a free hosted model from Groq, with a local option using Ollama."

What problem does it solve?

People ask the same policy questions again and again ("what is the refund policy?", "how many leave days?"). Answers are buried in long PDFs. Lumen answers instantly, shows where the answer came from so people can verify it, and says "I don't know" instead of guessing.

Part 2. Mental pictures (use these to explain)

Idea	Picture
Multi-tenant	An apartment building. One building (one running system), many flats (organizations). Each flat has its own key. Nobody can walk into another flat.
RAG (retrieval then answer)	An open-book exam. First the student finds the right pages in the book (retrieval), then writes the answer using only those pages (generation).
Embedding	A map of meaning. Every sentence gets a position on the map. Sentences with similar meaning land close together, so "money back" lands near "refund".

Idea	Picture
Three layers	A hospital. The receptionist (language model) talks to you. The lab machines (small models) produce the test results. The doctor's rules (application code) combine results into a diagnosis. The receptionist explains the diagnosis; she does not invent it.
JWT login token	A wristband at a festival. You show your ID once, you get a wristband, and every gate checks the band. It expires after 24 hours.
Assistant token	A guest pass for one room only. It lets you ask questions of one assistant. It cannot open the manager's office (the dashboard).
Groq vs Ollama	Renting a fast kitchen (Groq: someone else's powerful computers over the internet) versus cooking at home (Ollama: your own PC, free and private but slower).
Tunnel	A phone-forwarding service. Your PC has no public address, so the tunnel gives it one that forwards to your PC.

Part 3. How it works from start to finish

Step 1: an organization signs up

1. The React page sends `POST /api/auth/register` with organization name, email, password.
2. The server checks the email format and that the email is not already used.
3. The password is turned into a **bcrypt hash**. The real password is never stored.
4. A row is created in `organizations`. It gets a random **assistant token** (`ast_` plus a random string).
5. The server replies with a **JWT** (a signed token that says "this is organization 7", valid 24 hours). The browser keeps it in `localStorage`.

Step 2: they upload a PDF

1. The browser sends the PDF to `POST /api/kb/documents` with the JWT in the header.
2. The server finds the organization **from the token** (never from anything the browser sends).
3. It checks: is it a real PDF, at most 20 pages, at most 10 MB, and fewer than 10 documents.
4. The file is saved on disk under a **random name**, and a `documents` row is created with status `processing`. The server answers immediately.
5. In the **background**: the text is read page by page; each page is cut into small passages (about 650 characters, never across a page); each passage is turned into an **embedding** (384 numbers) and saved; rule patterns find sentences like "GPA of at least 3.5" and save them as **rule cards**; status becomes `ready`.
6. The dashboard checks every 1.2 seconds while anything is processing and shows the stage.

Step 3: a visitor asks a question

1. The chat page sends the question and the **assistant token**. No login.
2. The server finds the organization from the token.
3. **Router:** greetings, "thanks", gibberish, prompt-injection attempts and similar messages are answered by the app itself, instantly.
4. **Search:** only this organization's passages are loaded. Keyword search (BM25) and meaning search (embeddings) are merged, then a **reranker** scores each passage against the question.
5. **Guard:** if nothing is relevant enough, the assistant says it could not find it, and lists what it *can* help with. It does not guess.
6. **If the visitor states facts about themselves** ("I'm in year 2 with a 3.6 GPA"): facts are extracted; one yes/no check per rule runs in parallel; the code combines them (all must pass); missing facts are asked for.

7. **Answer:** the language model receives the verdict and the evidence, and writes a short, friendly answer. Guards fix any wording that contradicts the verdict or invents file names.
8. The answer **streams** to the browser word by word, together with the source document and page. The question and answer are saved in the history.

Part 4. Questions and answers

A. The project

Q1. Why did you build this? What was the task? Short answer: "It was a take-home assignment for a Senior Software Engineer role: build a multi-tenant AI Knowledge and Decision Assistant. Organizations upload PDFs, get a public assistant and embed code, and the AI must use more than one kind of decision, not just one prompt." Deeper: The assignment asked for registration and login, a PDF knowledge base with limits, a public assistant URL, an embeddable widget, answers with page references, at least two decision types combined, tenant isolation, error handling, a README, test data and a demo video.

Q2. How did you develop it? What was your process? Short answer: "I read the assignment carefully, listed the requirements, and planned the architecture first. Then I built it in layers: database and login, PDF ingestion, search, the decision pipeline, and the interface. After each part I wrote tests and tried real questions. Testing with real questions found many problems, which I fixed one by one." Deeper: Examples of problems found by testing: a $900 expense was assigned to the wrong approver because language models are weak with numbers, so I made the code decide; casual wording like "money back" did not find the refund policy, so I added a word-mapping step; the free AI service was rate-limiting, so I added automatic fallback to a second model. If they ask about tools: be honest. A good answer: "I used an AI coding assistant to speed up writing code, and I reviewed, tested and understood what it produced. I can walk you through any part."

Q3. How long did it take / what was the hardest part? Say the true time. For "hardest part": "Making the AI trustworthy. Getting an answer from a language model is easy. Making sure it never invents a rule, never contradicts the decision, and says 'I don't know' when it should, took most of the effort."

Q4. What would you do with more time? "A real database like Postgres with a vector extension, a background job queue for PDF processing, multiple users per organization with roles, a rule-review screen so admins can confirm what the system learned, feedback buttons on answers, and deployment on an always-on server."

B. Architecture

Q5. Explain the architecture. Short answer: "Three parts. A React frontend with an admin dashboard and the visitor chat. One FastAPI backend that exposes the API, serves the built website, runs PDF processing and runs the question engine. And a SQLite database for accounts, documents, passages, rules and history. The AI part has three layers: a language model for wording, small specialised models for decisions, and application code to combine them." Deeper: The backend has an upload pipeline (PDF to passages, embeddings and rules) and a question pipeline (router, search, classify, choose policy, extract facts, rule checks in parallel, combine, write answer, guards). Data flows only through the organization id.

Q6. Why is it one server for everything? "Simplicity. FastAPI serves both the API and the built React files, so there is one process, one port, and one thing to deploy. In production the website could move to a CDN and the API could scale separately."

Q7. Why FastAPI? "It is fast, async, validates requests automatically from type hints, generates API docs, and supports streaming responses, which I use to stream answers. It is also Python, where the AI and PDF libraries are."

Q8. Why React and Vite? "React is a standard choice for dashboards and chat UIs. Vite gives instant dev reload and a small, fast production build. I used Tailwind for styling and Motion for animation."

Q9. Why didn't you use LangChain or a framework? "The assignment did not require one, and a single custom pipeline is easier to explain, test and control. I wanted exact control over the safety checks. Fewer dependencies

also means fewer surprises."

Q10. What is RAG? "Retrieval Augmented Generation. First retrieve the passages most relevant to the question, then give only those to the language model so it answers from them. It is like an open-book exam. It reduces made-up answers and lets me show sources."

Q11. What is System 1? Why that name? "The assignment used the term for small, fast, specialised components that make narrow decisions, in contrast to a large slow language model. In my project they are: the embedding search and reranker, a question-type classifier, a policy chooser, and the rule checks."

C. Login, security and users

Q12. How does login work? Short answer: "Sign-up stores a bcrypt hash of the password. Login checks the password against the hash and returns a JWT, a signed token with the organization id and an expiry of 24 hours. The browser sends it in the `Authorization: Bearer` header on every admin request. A server function reads and verifies it and loads the organization." Deeper: HS256 signature with a secret from the environment. The token also carries `typ: admin`, so other kinds of tokens cannot be used. The organization id always comes from the verified token, never from the request body, so a user cannot ask for someone else's data.

Q13. Why bcrypt? "It is deliberately slow, so if the database leaked, guessing passwords would take very long. It also adds a random salt per password, so identical passwords look different."

Q14. Where do you store the JWT? Is that safe? "In the browser's localStorage. It is simple, but if the site had a cross-site scripting bug, a script could read it. The safer choice is an httpOnly cookie with CSRF protection. For this project I kept localStorage because there is no user-generated HTML on the dashboard, but I would move to cookies in production."

Q15. What is the assistant token? How is it different from the JWT? "It is the secret in the public link, like `ast_` plus 24 random bytes. It only identifies which organization's assistant to talk to. It is not a JWT, so it can never pass the admin check. If someone gets the link they can ask questions, but they cannot see documents or settings. The admin can regenerate it, and the old link stops working immediately."

Q16. Can one organization have multiple users? Honest answer: "Not yet. Right now one organization has one admin login: the email and password are on the organization record. That was enough for the assignment. To support multiple users I would add a `users` table with organization id, email, password hash and a role such as owner, editor or viewer. The token would carry the user id, organization id and role, and endpoints would check the role. Because every table and query is already organization-scoped, only the login layer changes."

Q17. How do you stop one organization seeing another's data? See Part 4-D, Q19. Short: every table has `org_id`, every query filters by it, and the organization comes from the verified token. I have tests that try cross-organization access.

Q18. What other security measures did you add? "Passwords are hashed. Secrets come from environment variables, not code. Uploaded PDFs are saved under random names and are never served publicly: downloading needs the admin login. There are upload limits and real PDF validation. Rate limiting is 30 questions a minute per assistant and visitor address. Prompt-injection messages are caught by the router. The assistant has no tools or actions, so a successful trick cannot delete or change anything. Errors return friendly messages and not stack traces."

D. Multi-tenancy and data

Q19. Explain multi-tenancy. How is it implemented? Short answer: "Shared database, shared tables, with an organization id on every row. Every query filters by that id, at the database level, so another organization's text is never even loaded. The organization comes from the signed token for admins, or from the assistant token for visitors. I wrote tests that try to break it." Deeper: The three common approaches are shared tables with a tenant column (what I used), a separate schema per tenant, and a separate database per tenant. Shared tables are cheapest and simplest to run. Separate databases give the strongest isolation but cost more to manage. For stronger guarantees on Postgres I would add row-level security, so even a coding mistake cannot leak rows.

Q20. How do you prove the isolation works? "Automated tests. They register two organizations, upload different documents, and check that organization B's assistant cannot find organization A's content, that B's admin token cannot read A's history or documents, that the assistant token cannot call admin APIs, and that replies only list the asking organization's documents. On screen I also ask Acme about a scholarship and it cannot find it."

Q21. What is in the database? "Five tables: `organizations` (the tenants and their assistant settings), `documents` (each PDF's metadata and status), `chunks` (the text passages with their embeddings), `rules` (rule cards mined from the PDFs), and `question_logs` (every question, the answer, sources and timing). Every table except organizations has an `org_id` foreign key with cascade delete."

Q22. Why SQLite? Is it good enough? "SQLite is zero-setup, one file, perfect for a take-home and a demo. It allows one writer at a time, so it does not suit heavy concurrent writes. Because I used SQLAlchemy, switching to Postgres is a settings change (`DATABASE_URL`) plus a proper migration tool."

Q23. How are embeddings stored and searched? "Each passage's 384 numbers are saved as bytes in the `chunks` table. At question time I load only that organization's passages and compare them to the question vector with a fast matrix multiplication in numpy. With at most ten documents of twenty pages that is a few hundred vectors, so brute force is instant. At large scale I would use a vector index such as pgvector."

Q24. How is data managed when a document is replaced or deleted? "The old file is removed from disk and its passages and rules are deleted (cascade). A replace runs the same processing pipeline again. If the new file fails validation, the user sees a clear message."

Q25. What happens if processing a PDF fails? "The document status becomes `failed` with a readable reason, the dashboard shows it, and nothing crashes. The user can delete or replace it. This is covered by a test."

Q26. How are database changes (migrations) handled? "At startup, `init_db` creates missing tables and has a small built-in step that adds new columns to older SQLite databases. I used it to add stored answers to the history without losing data. For production I would use Alembic."

E. The AI

Q27. How does the AI part work? Short answer: "The language model is used only for language: it understands the question wording, extracts facts, and writes the answer. The actual decisions are made by small models and code. So the verdict, like 'not eligible', is computed by rules, not by the language model's opinion."

Q28. Walk me through one multi-decision question. Use: "I'm a second-year student with a 3.6 GPA and I already have a sports scholarship. Am I eligible for the Merit Scholarship?"

1. **Search** finds the Merit Scholarship passages for this organization.
2. **Classifier** labels it an eligibility question.
3. **Policy chooser** picks Merit Scholarship.
4. **Facts** are extracted: year 2, GPA 3.6, has a sports scholarship. A fact the model invents is thrown away if it is not in the question.
5. **Rule checks run in parallel:** year at least 2 passes, GPA at least 3.5 passes, no sports scholarship fails. An evidence score runs at the same time.
6. **Combine:** all must pass, so the result is not eligible.

7. **Answer:** the language model words the result. The reasons are written by the app, and a guard rejects wording that contradicts the verdict.

If the visitor gave no facts, the result is "need information" and it asks for the missing ones instead of guessing.

Q29. Why not just send everything to the language model in one prompt? "Because it is unreliable for decisions. It can invent conditions, miscalculate numbers, and contradict itself, and you cannot audit why. With separate decisions, each step is testable, the rule checks are deterministic, and I can show the user exactly which condition failed and on which page."

Q30. What are the "decision types"? "The assignment asked for at least two. I used four. **Classification:** question type (logistic regression). **Choice:** which policy applies, and which approval tier fits an amount. **Boolean:** does each rule pass, fail, or is it unknown. **Score:** how well the evidence supports an answer (the reranker)."

Q31. Did you train a model? Honest answer: "I did not fine-tune the language model. The embedding model and the reranker are pre-trained and used as they are. The only thing I train is a small question-type classifier, a logistic regression trained at startup on about 40 example questions. The rules are found with patterns, not learned. So the quality comes from the retrieval, rules, prompts and safety checks, not from training."

Q32. How do you prevent hallucinations? "Several layers. Answers are written only from retrieved passages. If nothing relevant is found the assistant refuses. Facts the model extracts are checked against the question text. The verdict comes from code, and a guard rejects wording that contradicts it. The explanation sentences are written by the app. File names in citations are snapped to real documents. And every answer shows its source page so the user can verify."

Q33. What is the reranker? "After the first search returns candidates, a reranker reads the question and each passage together and gives a score for how well the passage answers it. It is more accurate than comparing separate vectors, but slower, so I only use it on the top candidates."

Q34. What is hybrid search? "Keyword search (BM25) is good at exact words like names and numbers. Embedding search is good at meaning, like 'money back' versus 'refund'. I run both and merge the rankings with Reciprocal Rank Fusion, then rerank."

Q35. How did you handle everyday wording like "money back if I quit"? "A small mapping step adds the policy words to the search query only: 'money back' adds 'refund', 'quit' adds 'withdraw'. It only widens the search; the answer is still written only from passages that were found."

Q36. How do you handle prompt injection? "A router checks for phrases like 'ignore previous instructions' or 'reveal your prompt' and refuses before any model sees them. The assistant also has no tools, so even a successful trick cannot change data. The honest limit: a regex list can be bypassed by clever wording, so in production I would add a classifier-based guard and output filtering."

Q37. What are the types of answer the assistant gives? "Direct facts, policy explanations, step lists, comparisons, eligibility decisions, 'I need more information', and 'I could not find that', which also lists what it can help with."

Q38. How does it handle follow-up questions? "The browser sends the earlier question and the last few turns. The earlier question is added to the search, so 'what about third year?' works. The recent turns go to the language model only to understand references, never as a source of facts. If someone says 'what?' or 'I don't understand', the previous answer is explained again in simpler words."

F. Ollama, Groq and the language model

Q39. What is Ollama? What is Groq? Short answer: "Both are ways to run a language model. **Ollama** runs open models on your own computer. It is free and private, but slow on a CPU, about 5 to 15 seconds per answer. **Groq** is a company that hosts open models on very fast hardware and gives an API with a free tier. It is fast and more capable, but it needs internet and an API key, and the free tier has limits."

Q40. Why did you support both? "The brief said the solution should be free to run. Ollama needs no account and no key, so anyone can run it offline. Groq gives a better demo because it is fast. A single environment variable, `LLM_PROVIDER`, switches between them with no code change. There is also a mock mode for tests."

Q41. How is the model connected? "Through `llm.py`. For Ollama it calls the local server at port 11434. For Groq it uses the OpenAI-style chat API, so the same code also works for any compatible service. Answers are streamed token by token so the user sees text immediately."

Q42. What happens if the model is down or rate-limited? "First the app tries fallback models in order, because on free tiers every model has its own allowance. If all fail, the app still answers using a template built from the best matching sentence or the decision result, and marks the answer as degraded. So the product never shows a blank error for a model problem."

Q43. Which model did you use and why? "For the live demo, `qwen/qwen3.8-27b` on Groq, with `gpt-oss` models as fallbacks, because they were free, fast and available on my account. Locally, `qwen2.5:3b` through Ollama. The model names are settings, not code, so they are easy to change. Names on free services change, which is why failover matters."

Q44. What did you learn about Groq's free tier? "It limits tokens per minute for each model, about 8,000 for the one I used. Heavy testing hit it and answers fell back to plain text. I fixed it with fallback models and by sending fewer, better passages to the model."

Q45. Does my document data go to Groq? Is that a privacy problem? "Only the few passages retrieved for a question are sent, not whole documents, but yes, that text goes to the provider. For sensitive organizations I would use a local model through Ollama or a self-hosted model, which keeps data inside. The choice is one setting."

G. Backend implementation

Q46. How is the backend organised? "`main.py` has the endpoints; `db.py` the tables; `security.py` login; `pdfproc.py` and `ingest.py` the upload pipeline; `retrieval.py` search; `engine.py` the question pipeline; `rules.py` rule mining and checks; `classifier.py`; `smalltalk.py` the router; `suggest.py` suggestions; `llm.py` the model client; `query.py` word mapping."

Q47. How does streaming work? "The endpoint `/api/ask/stream` returns Server-Sent Events. The engine is an async generator that yields events as it works: a step started or finished, a piece of text, a replacement if a guard corrected the answer, and a final result. The browser reads the stream and updates the screen live. That is also how the 'How I decided' panel animates."

Q48. Is it asynchronous? How do you handle slow work? "The endpoints are async. Slow CPU work like embeddings and reranking runs in worker threads so it does not block other requests. PDF processing runs in the background after the upload request returns, using FastAPI background tasks. For large scale I would use a real job queue."

Q49. How do you handle errors? "Validation errors return clear 4xx messages. A failed PDF is marked failed with a reason. A model failure falls back to a template answer. A decision-model failure returns a clear message. Unexpected errors return a generic friendly message without internal details. All of these paths have tests."

Q50. How do you limit abuse? "A rate limit of 30 questions per minute per assistant token and visitor address, upload limits, a question length limit, and the router for abusive or junk messages. The limiter is in memory, so for many servers I would move it to Redis."

H. Frontend

Q51. How is the frontend built? "React single-page app with React Router. Pages: landing, sign in and sign up, the dashboard with four tabs, and the public chat. A small API file wraps `fetch` and adds the login token. A context shares loaded data across dashboard tabs. Tailwind and CSS variables handle light and dark themes."

Q52. How does the embeddable widget work? "The customer pastes one script tag with the assistant token. The script adds a floating button, and clicking it opens an iframe pointing at the public chat page with `?embed=1`. The script only knows the public token, so it cannot reach anything private. There is also a plain iframe option."

Q53. How does the live preview work? "It is the real chat component. The settings form passes its unsaved values (name, colour, greeting, suggestions) into it as props, so you see changes as you type, before saving."

Q54. What did you do for user experience? "Inline validation, a password strength bar, upload progress, processing stages, empty and error states, a guided tour with skip, starter questions, follow-up buttons, a full question history with search, and friendly replies for casual messages. It works on phones and in dark mode."

I. Hosting and deployment

Q55. How is it hosted? Short answer: "It runs on my own computer. FastAPI serves both the API and the built website on port 8000, and a free Cloudflare quick tunnel gives it a public address that forwards to my PC. So anyone with the link can use it while my PC is on." Deeper: I chose this because free cloud tiers were too small. The app loads two machine-learning models and needs about 550 MB while processing a PDF, but free hosts give 512 MB. Hugging Face now charges for Docker spaces. A small cloud VM with a free tier that stays online would be the next step.

Q56. Why not deploy to a normal cloud? "The assignment said deployment is optional. Free tiers did not fit the memory the models need, or needed a card. I chose a free setup that works today and documented how to move to a proper server."

Q57. What does a production deployment look like? "A container image running several app instances behind a load balancer with HTTPS; Postgres with a vector extension; object storage such as S3 for PDFs; a job queue and workers for ingestion; Redis for rate limiting and caching; secrets in a secret manager; logging and monitoring; and a managed or self-hosted language model. The Dockerfile in the project is the starting point."

Q58. How does the Docker setup work? "The Dockerfile builds the React app in one stage, then builds a Python image that contains the backend, the built website, the sample data and the downloaded models. `docker-compose` also starts an Ollama container and downloads the model, so one command, `docker compose up --build`, runs everything with no account or key."

Q59. How do you update the live site? "Run the update script. It rebuilds the frontend, stops the old server, starts the new one and waits until it is healthy. The tunnel keeps running, so the link stays the same. Then I save the change to GitHub."

J. Testing and quality

Q60. How did you test it? "Three levels. **119 automated tests** run offline using a fake embedding and a mock model, so they are fast and repeatable; they cover login, uploads, limits, decisions, routing, history and tenant isolation. **A 19-question evaluation** against the real models, covering direct, policy, procedural, multi-condition, comparison and no-answer questions across two organizations. **A battery of 44 scenarios** with typos, capital letters, emoji, off-topic questions and cross-organization probes."

Q61. How do you know the answers are correct? "I cannot guarantee it, and I do not claim to. What I do: ground every answer in retrieved text, show the source page so users can check, test with a fixed question set including expected decisions, and refuse when evidence is weak. In production I would add feedback buttons and review the 'not found' and thumbs-down questions to improve the documents and rules."

Q62. What bug are you proudest of finding? Pick one: "A wrong approver for a $900 expense. The model said Finance Committee, but the policy says Director. Models are unreliable at comparing numbers, so I moved that choice into code and the model only words it." Or: "The AI service was silently failing because I configured a model name my account did not have. A test I wrote exposed that every answer was falling back to plain text."

Part 5. Hard questions and honest answers

"Is this really AI, or just search?" "Both. Search finds passages. A language model understands the question and writes the answer. Small models classify the question and score evidence, and rules make the decision. The AI parts are real, but the key design choice is that the language model does not decide alone."

"What if two organizations upload the same file?" "Each gets its own copy, its own passages and its own embeddings, all tagged with its own organization id. They never share rows."

"What if the PDF is a scanned image?" "It will fail with a clear message because there is no text to read. I did not add OCR. I would add OCR with a library like Tesseract."

"Can the rules be wrong?" "Yes. Rules are found by patterns, so unusual wording can be missed or misread. That is why each rule keeps its evidence sentence and page, and why I list a rule-review screen as the next improvement."

"What about OR conditions?" "Currently rules combine with AND only. Real policies sometimes say 'either X or Y'. I would extend rule cards with groups and operators."

"How would it scale to 10,000 organizations?" "Move to Postgres with pgvector and indexes on `org_id`; put PDFs in object storage; process uploads in a queue with workers; run several stateless app instances; move rate limiting and caching to Redis; use a hosted or self-hosted model with proper capacity; and add monitoring and per-tenant quotas."

"What are the security weaknesses?" "The token is in localStorage; the router for injection is pattern-based; CORS is open on public endpoints by design; the rate limiter is per server; there is one admin per organization; and the free tunnel has no uptime guarantee. I know the fixes for each."

"Why should we trust the live link?" "It is a demo from my machine through a secure tunnel, and the code and tests are in GitHub. For a real launch I would deploy to a proper server."

"What would you do differently?" "Start with Postgres, add multiple users per organization from the beginning, build the rule-review screen earlier, and host it on a small always-on server."

"Did you use AI tools to build this?" Be honest. Example: "Yes, I used an AI coding assistant to speed up writing code. I made the design decisions, reviewed everything, wrote and ran the tests, found and fixed issues from real usage, and I can explain every part."

Part 6. Things to say when you do not know something

- "I did not build that part. This is how I would approach it: ..."
- "That is a limit of the current version. The fix would be ..."
- "Good question. Let me check the code and show you." (Then open the file.)
- Do not invent numbers. If you forget one, say "I would need to check, but it is roughly ..." and use the cheat sheet below.

Part 7. Files to open before the interview (read in this order, 10 minutes each)

1. `backend/app/main.py`: all endpoints. Find `register`, `login`, `upload_document`, `ask_stream`.
2. `backend/app/security.py`: password hashing, JWT creation, `current_org`.
3. `backend/app/db.py`: the five tables.
4. `backend/app/ingest.py` and `pdfproc.py`: the upload pipeline.
5. `backend/app/retrieval.py`: the search.

6. `backend/app/engine.py`: the question pipeline. Read `run()` from top to bottom, and `combine`.
7. `backend/app/rules.py`: rule mining and `evaluate`.
8. `frontend/src/components/ChatPanel.jsx`: the chat and streaming.
9. `backend/tests/test_isolation.py`: the isolation proof.

Part 8. Cheat sheet (numbers and names)

Item	Value
Backend	Python 3.12, FastAPI, SQLAlchemy 2, SQLite
Frontend	React 19, Vite, Tailwind 4, React Router, Motion
Embedding model	`BAAI/bge-small-en-v1.5`, 384 numbers per passage
Reranker	`Xenova/ms-marco-MiniLM-L-6-v2` (cross-encoder)
Classifier	logistic regression, about 40 example questions, 5 types
Language model (live)	Groq, `qwen/qwen3.8-27b`; fallbacks `openai/gpt-oss-20b`, `openai/gpt-oss-120b`
Language model (local)	Ollama, `qwen2.5:3b`
Search	BM25 plus embeddings, merged with Reciprocal Rank Fusion (k=60), reranked, top 5
Passage size	up to about 650 characters, never across a page
Refusal threshold	evidence score below 0.12
Login	bcrypt password hash, JWT (HS256), 24 hours
Assistant token	`ast_` plus 24 random bytes
Upload limits	10 PDFs, 20 pages, 10 MB each
Rate limit	30 questions a minute per assistant and visitor address
Tables	organizations, documents, chunks, rules, question_logs
Tests	119 unit and API tests, 19 evaluation questions, 44 scenarios
Demo accounts	`demo-northfield@example.com` / `demo-password-1`; `demo-acme@example.com` / `demo-password-2`
Repository	github.com/salman-kv/lumen
Full technical docs	`docs/Lumen_Documentation.pdf`